D'AUTRES TYPES CONSTRUITS

Les tableaux - part. 2: construire des tableaux en compréhension

Construction de tableaux en compréhension

Supposons le problème suivant:

Construire un tableau des 10 premiers entiers naturels multiples de 9.

On peut construire ce tableau comme on l'a vu lors de la découverte des tableaux (on dit aussi construire en **extension**):

```
t = [0, 9, 18, 27, ...]
```

C'est potentiellement long et pas très élégant! Une deuxième solution serait d'utiliser une boucle `for` :

```
t = [0] * 10
for i in range(10):
    t[i] = 9 * i
```

Python propose une syntaxe plus compacte pour réaliser cette opération. Il s'agit de la **construction par compréhension** (on dit aussi en *intension*):

```
t = [9 * i for i in range(10)]
```

À retenir

Plus généralement, au lieu de déclarer un tableau puis de le remplir avec une boucle, on préfèrera une construction plus concise appelée construction par compréhension, qui prend la forme suivante:

```
tableau = [expr(i) for i in sequence]
```

`expr(i)` est une *expression* de la variable `i` qui parcourt tous les éléments de `sequence` (qui peut être un objet `range`, un tableau, une chaine de caractère ou un des types qui seront vus dans les paragraphes ci-après).

Modifions notre consigne de départ:

Parmi les 10 premiers entiers naturels multiples de 9, construire un tableau des entiers pairs uniquement.

La construction par compréhension permet là encore, une écriture compacte:

```
t = [9 * i for i in range(10) if 9 * i % 2 == 0]
```

À retenir

La construction par compréhension permet d'ajouter une condition booléenne.

```
tableau = [expr(i) for i in sequence if condition]
```

`condition` est exprimée avec les opérateurs de comparaisons habituels.

```
# Exemple
t = [9 * i for i in range(20) if 9 * i % 2 == 0]
print(t)
```

```
[0, 18, 36, 54, 72, 90, 108, 126, 144, 162]
```

Complément: agrandir un tableau

Le langage python autorise un traitement particulier sur un tableau: pouvoir **l'agrandir sur sa droite**, même si sa taille a déjà été fixée. La syntaxe fait intervenir une fonction disponible avec les objets de type *list* (on dit aussi une *méthode*): `append`.

```
tab = [1, 2, 3]
tab.append(5)
print(tab)
```

```
[1, 2, 3, 5]
```

Attention

Dans le cadre du programme de 1re NSI, on privilégiera autant que possible la compréhension de tableau par rapport à l'utilisation de la méthode `append`.

Les p-uplets

Construction et propriétés d'un p-uplet

p-uplet

En mathématique, un **p-uplet** est une collection ordonnée d'objets.

Python fournit un type adapté à la représentation des p-uplets: le type `tuple`. On créé un tuple en écrivant les composantes du p-uplet **séparées par une virgule** et de préférence **entre parenthèses**:

```
notes = (17, 15, 16)
```

Comme avec les tableaux, on peut obtenir la taille d'un tuple avec la fonction native `len` et accéder au $i^{ème}$ élément avec la notation habituelle des crochets:

```
notes = (17, 15, 16, 10)
print("Taille du p-uplet: ", len(notes))
print("Valeur du 3ème élément (index 2): ", notes[2])
```

```
Taille du p-uplet:  4
Valeur du 3ème élément (index 2):  16
```

```
notes[1] = 7
print(notes)
```

```
---------------------------------------------------------------------------

TypeError                                 Traceback (most recent call last)

<ipython-input-8-c340dc6cd5d2> in <module>
----> 1 notes[1] = 7
      2 print(notes)


TypeError: 'tuple' object does not support item assignment
```

Contrairement aux tableaux les composantes d'un tuple ne sont pas modifiables. On dit que les tuples sont des objets immuables (ou non mutables).

```
notes[3] = 16.5
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-4-06e2d03882e0> in <module>()
----> 1 notes[3] = 16.5

TypeError: 'tuple' object does not support item assignment
```

Parcourir un p-uplet

Les méthodes de parcours rencontrées lors de la découverte des tableaux sont toujours valables. On peut itérer sur les index ou sur les éléments.

```
print("****Relevé de notes****")
for n in notes:
    print(n)
print("****Moyenne****")
s = 0
for i in range(len(notes)):
    s = s + notes[i]
m = s / len(notes)
print(m)
```

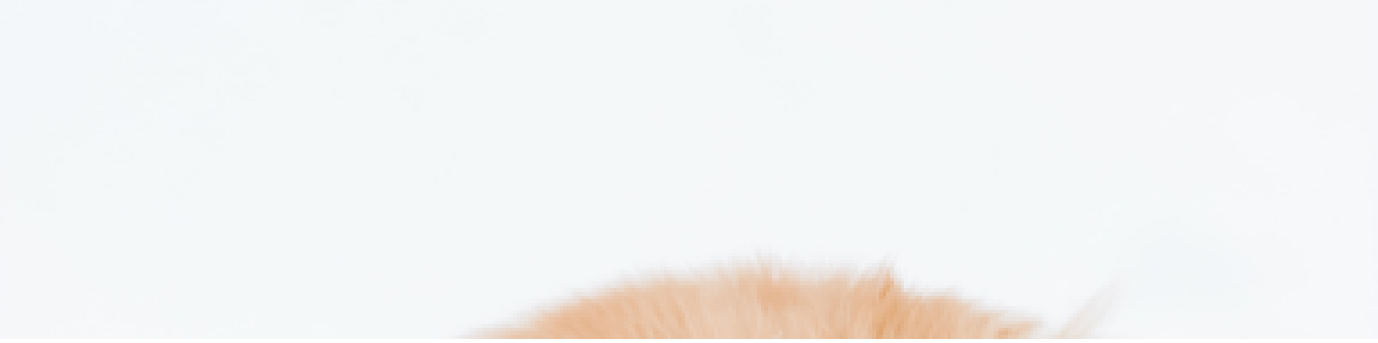
```
****Relevé de notes****
17
15
16
10
****Moyenne****
14.5
```

Insuffisance des p-uplets

Etant donné le problème suivant:

On souhaite stocker les informations concernant des fichers image dans un p-uplet. Ce dernier aura pour composante sa largeur en pixel, sa hauteur en pixel et sa taille en kilo-octet.

Par exemple, soit le fichier "dog.png" qui a une largeur de 384 pixels, une hauteur de 576 pixels et dont la taille est de 243 ko.



On peut le modéliser par un simple tuple:

```
image = (384, 576, 243)
```

Cependant cette solution n'est pas idéale. En effet, il faut se souvenir de l'ordre dans lequel les entiers ont été rentrés !
On évitera d'utiliser des p-uplets lorsque des ambiguités ou confusions sont possibles.

Une solution possible

Pour contourner le problème soulevé précédemment, Python a une solution: les **p-uplets nommés**. Chaque composante du p-uplet se voit donner un nom!

Python possède un type permettant de manipuler les p-uplets nommés: les `namedtuple` du module `collections`. Cependant, et conformément au programme, en classe de 1re NSI, **on utilisera des dictionnaires pour implémenter des p-uplets nommés**.

Les dictionnaires

On consultera au préalable la vidéo suivante.

Construction et propriétés des dictionnaires

Python possède une structure de données très importante: **le dictionnaire**. Contrairement aux tableaux ou aux tuples, les **valeurs** ne sont plus associées à des index forcément numériques mais à des **clés** qui peuvent être des nombres, des chaines de caractères, des tuples. On déclare un dictionnaire en plaçant ses paires clé-valeur entre accolades et séparées par une virgule. La clé et la valeur étant séparées par le caractère `:`.

Ainsi, le problème du fichier image précédent peut être modélisé par:

```
image = {'nom':'dog.png', 'larg': 384, 'haut': 576, 'taille': 243}
print("Taille de l'image:", image['taille'], "ko")
print("Hauteur:", image['haut'], "pixels")
```

```
Taille de l'image: 243 ko
Hauteur: 576 pixels
```

Autre exemple
On peut également construire un dictionnaire à partir d'un dictionnaire vide. La syntaxe est la suivante:

```
cyclones_reunion = {}
cyclones_reunion["bejisa"] = (2, 1, 2014)
cyclones_reunion["dumile"] = (3, 1, 2013)
cyclones_reunion["dina"] = (22, 1, 2002)
cyclones_reunion["gamede"] = (25, 2, 2007)
print(cyclones_reunion)
```

```
{'bejisa': (2, 1, 2014), 'dumile': (3, 1, 2013), 'dina': (22, 1, 2002), 'gamede': (25, 2, 2007)}
```

Le nombre d'entrées du dictionnaire est obtenu avec la fonction `len`.

```
len(cyclones_reunion)
```

```
4
```

On accède à la **valeur** associée à une **clé** d'un dictionnaire `d` avec une construction du type `d[clé]`. Si la clé n'existe pas, python lèvera une exception **KeyError**.

```
print(cyclones_reunion["dina"])
```

```
(22, 1, 2002)
```

Si la clé n'existe pas:

```
print(cyclones_reunion["carlos"])
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-9-0306c0a2304c> in <module>()
----> 1 print(cyclones_reunion["carlos"])

KeyError: 'carlos'
```

Enfin, **un dictionnaire n'est pas ordonné**. Les éléments ne sont pas forcément afichés dans l'ordre de leur création.

Parcours d'un dictionnaire

L'appartenance à un dictionnaire (*en réalité aux clés d'un dictionnaire*) se fait avec le mot clé `in`. Ainsi:

```
'dina' in cyclones_reunion.keys()
```

```
True
```

```
'carlos' in cyclones_reunion.keys()
```

```
False
```

Le parcours d'un dictionnaire `d` peut se faire en itérant avec une boucle `for` sur:

- les clés

```
for cle in d.keys(): # souvent abrégé en `for cle in d`
    ...
```

- les valeurs

```
for val in d.values():
    ...
```

- les couples (clé,valeur)

```
for cle,val in d.items():
    ...
```

Résumé

Tableau

On peut itérer directement sur les éléments d'un tableau avec la construction du type `for element in tableau:`.
Pour remplir un tableau on peut utiliser une construction par compréhension:

```
tableau = [expr(x) for x in sequence]
```

p-uplet

Les p-uplets sont implémentés en python par le type `tuple`. Il s'agit d'un objet non mutable.
La syntaxe pour construire un p-uplet ayant N composantes, est:

```
mon_objet = (comp1, comp2, ..., compN)
```

Les différentes composantes sont accessibles par indexation, comme pour un tableau: `mon_objet[0]`, `mob_objet[1]` etc.
Le parcours est identique à celui d'un tableau.

Dictionnaires

Un dictionnaire est une structure regroupant des paires "clé:valeur". En python, ces paires sont situées entre accolades et séparées par une virgule.

```
image = {'nom':'dog.png', 'larg': 384, 'haut': 576, 'taille': 243}
```

On peut itérer sur les clés, les valeurs ou les paires clé,valeur avec les méthodes `keys()`, `values()` ou `items()`.

p-uplets nommés

En python, les p-uplets nommés sont implémentés avec des dictionnaires.